

小结

78

类型是 C++ 编程的基础。

类型规定了其对象的存储要求和所能执行的操作。C++ 语言提供了一套基础内置类型，如 `int` 和 `char` 等，这些类型与实现它们的机器硬件密切相关。类型分为非常量和常量，一个常量对象必须初始化，而且一旦初始化其值就不能再改变。此外，还可以定义复合类型，如指针和引用等。复合类型的定义以其他类型为基础。

C++ 语言允许用户以类的形式自定义类型。C++ 库通过类提供了一套高级抽象类型，如输入输出和 `string` 等。

术语表

地址 (address) 是一个数字，根据它可以找到内存中的一个字节。

别名声明 (alias declaration) 为另外一种类型定义一个同义词：使用“名字=类型”的格式将名字作为该类型的同义词。

算术类型 (arithmetic type) 布尔值、字符、整数、浮点数等内置类型。

数组 (array) 是一种数据结构，存放着一组未命名的对象，可以通过索引来访问这些对象。3.5 节将详细介绍数组的知识。

auto 是一个类型说明符，通过变量的初始值来推断变量的类型。

基本类型 (base type) 是类型说明符，可用 `const` 修饰，在声明语句中位于声明符之前。基本类型提供了最常见的数据类型，以此为基础构建声明符。

绑定 (bind) 令某个名字与给定的实体关联在一起，使用该名字也就是使用该实体。例如，引用就是将某个名字与某个对象绑定在一起。

字节 (byte) 内存中可寻址的最小单元，大多数机器的字节占 8 位。

类成员 (class member) 类的组成部分。

复合类型 (compound type) 是一种类型，它的定义以其他类型为基础。

const 是一种类型修饰符，用于说明永不改变的对象。`const` 对象一旦定义就无法再

赋新值，所以必须初始化。

常量指针 (const pointer) 是一种指针，它的值永不改变。

常量引用 (const reference) 是一种习惯叫法，含义是指向常量的引用。

常量表达式 (const expression) 能在编译时计算并获取结果的表达式。

constexpr 是一种函数，用于代表一条常量表达式。6.5.2 节（第 214 页）将介绍 `constexpr` 函数。

转换 (conversion) 一种类型的值转变成另外一种类型值的过程。C++ 语言支持内置类型之间的转换。

数据成员 (data member) 组成对象的数据元素，类的每个对象都有类的数据成员的一份拷贝。数据成员可以在类内部声明的同时初始化。

声明 (declaration) 声称存在一个变量、函数或是别处定义的类型。名字必须在定义或声明之后才能使用。

声明符 (declarator) 是声明的一部分，包括被定义的名字和类型修饰符，其中类型修饰符可以有也可以没有。

decltype 是一个类型说明符，从变量或表达式推断得到类型。

默认初始化 (default initialization) 当对象未被显式地赋予初始值时执行的初始化行

79

为。由类本身负责执行的类对象的初始化行为。全局作用域的内置类型对象初始化为 0；局部作用域的对象未被初始化即拥有未定义的值。

定义 (definition) 为某一特定类型的变量申请存储空间，可以选择初始化该变量。名字必须在定义或声明之后才能使用。

转义序列 (escape sequence) 字符特别是那些不可打印字符的替代形式。转义以反斜线开头，后面紧跟一个字符，或者不多于 3 个八进制数字，或者字母 x 加上 1 个十六进制数。

全局作用域 (global scope) 位于其他所有作用域之外的作用域。

头文件保护符 (header guard) 使用预处理变量以防止头文件被某个文件重复包含。

标识符 (identifier) 组成名字的字符序列，标识符对大小写敏感。

类内初始值 (in-class initializer) 在声明类的数据成员时同时提供的初始值，必须置于等号右侧或花括号内。

在作用域内 (in scope) 名字在当前作用域内可见。

被初始化 (initialized) 变量在定义的同时被赋予初始值，变量一般都应该被初始化。

内层作用域 (inner scope) 嵌套在其他作用域之内的作用域。

整型 (integral type) 参见算术类型。

列表初始化 (list initialization) 利用花括号把一个或多个初始值放在一起的初始化形式。

字面值 (literal) 是一个不能改变的值，如数字、字符、字符串等。单引号内的是字符字面值，双引号内的是字符串字面值。

局部作用域 (local scope) 是块作用域的习惯叫法。

底层 const (low-level const) 一个不属于顶层的 const，类型如果由底层常量定义，

则不能被忽略。

成员 (member) 类的组成部分。

不可打印字符 (nonprintable character) 不具有可见形式的字符，如控制符、退格、换行符等。

空指针 (null pointer) 值为 0 的指针，空指针合法但是不指向任何对象。

nullptr 是表示空指针的字面值常量。

对象 (object) 是内存的一块区域，具有某种类型，变量是命名了的对象。

外层作用域 (outer scope) 嵌套着别的作用域的作用域。

指针 (pointer) 是一个对象，存放着某个对象的地址，或者某个对象存储区域之后的下一地址，或者 0。

指向常量的指针 (pointer to const) 是一个指针，存放着某个常量对象的地址。指向常量的指针不能用来改变它所指对象的值。

预处理器 (preprocessor) 在 C++ 编译过程中执行的一段程序。

预处理变量 (preprocessor variable) 由预处理器管理的变量。在程序编译之前，预处理器负责将程序中的预处理变量替换成它的真实值。

引用 (reference) 是某个对象的别名。

80 **对常量的引用 (reference to const)** 是一个引用，不能用来改变它所绑定对象的值。对常量的引用可以绑定常量对象，或者非常量对象，或者表达式的结果。

作用域 (scope) 是程序的一部分，在其中某些名字有意义。C++ 有几级作用域：

全局 (global) ——名字定义在所有其他作用域之外。

类 (class) ——名字定义在类内部。

命名空间 (namespace) ——名字定义在命名空间内部。

块 (block) ——名字定义在块内部。

名字从声明位置开始直至声明语句所在的作用域末端为止都是可用的。

分离式编译 (separate compilation) 把程序分割为多个单独文件的能力。

带符号类型 (signed) 保存正数、负数或 0 的整型。

字符串 (string) 是一种库类型，表示可变长字符序列。

struct 是一个关键字，用于定义类。

临时值 (temporary) 编译器在计算表达式结果时创建的无名对象。为某表达式创建了一个临时值，则此临时值将一直存在直到包含有该表达式的最大的表达式计算完成为止。

顶层 const (top-level const) 是一个 const，规定某对象的值不能改变。

类型别名 (type alias) 是一个名字，是另外一个类型的同义词，通过关键字 typedef 或别名声明语句来定义。

类型检查 (type checking) 是一个过程，编译器检查程序使用某给定类型对象的方式与该类型的定义是否一致。

类型说明符 (type specifier) 类型的名字。

typedef 为某类型定义一个别名。当关键字 typedef 作为声明的基本类型出现时，声明中定义的名字就是类型名。

未定义 (undefined) 即 C++ 语言没有明确规定的情况。不论是否有意为之，未定义行为都可能引发难以追踪的运行时错误、安全性和可移植性问题。

未初始化 (uninitialized) 变量已定义但未被赋予初始值。一般来说，试图访问未初始化变量的值将引发未定义行为。

无符号类型 (unsigned) 保存大于等于 0 的整型。

变量 (variable) 命名的对象或引用。C++ 语言要求变量要先声明后使用。

void* 可以指向任意非常量的指针类型，不能执行解引用操作。

void 类型 是一种有特殊用处的类型，既无操作也无值。不能定义一个 void 类型的变量。

字 (word) 在指定机器上进行整数运算的自然单位。一般来说，字的空间足够存放地址。32 位机器上的字通常占据 4 个字节。

&运算符 (& operator) 取地址运算符。

***运算符 (* operator)** 解引用运算符。解引用一个指针将返回该指针所指的对象，为解引用的结果赋值也就是为指针所指的對象赋值。

#define 是一条预处理指令，用于定义一个预处理变量。

#endif 是一条预处理指令，用于结束一个 #ifdef 或 #ifndef 区域。

#ifdef 是一条预处理指令，用于判断给定的变量是否已经定义。

#ifndef 是一条预处理指令，用于判断给定的变量是否尚未定义。

第 3 章

字符串、向量和数组

内容

3.1 命名空间的 using 声明	74
3.2 标准库类型 string	75
3.3 标准库类型 vector	86
3.4 迭代器介绍	95
3.5 数组	101
3.6 多维数组	112
小结	117
术语表	117

除了第 2 章介绍的内置类型之外, C++ 语言还定义了一个内容丰富的抽象数据类型库。其中, string 和 vector 是两种最重要的标准库类型, 前者支持可变长字符串, 后者则表示可变长的集合。还有一种标准库类型是迭代器, 它是 string 和 vector 的配套类型, 常被用于访问 string 中的字符或 vector 中的元素。

内置数组是一种更基础的类型, string 和 vector 都是对它的某种抽象。本章将分别介绍数组以及标准库类型 string 和 vector。